
SISTEMA DE GESTION DE CATALOGO DE LIBROS

202400898 – Angel Adrián Rodríguez Barrientos

Resumen

El presente informe detalla el diseño y desarrollo de un sistema de gestión y control implementado en el lenguaje de programación C# para la administración del catálogo de libros y categorías de una biblioteca. Mediante el uso del modelo de Programación Orientada a Objetos y el patrón Modelo-Vista-Controlador (MVC) en ASP.NET Core, se gestionan los datos provenientes de archivos de formato XML para construir la jerarquía de categorías utilizando un Árbol N-ario basado en la representación binaria de Primer Hijo / Siguierte Hermano, sin usar las colecciones nativas del lenguaje. Para la organización, consulta y mantenimiento de los libros dentro de cada categoría, se implementó una estructura de Árbol AVL que garantiza búsquedas, inserciones y eliminaciones eficientes mediante rotaciones de balanceo. Finalmente, el sistema genera reportes visuales en código DOT para la herramienta Graphviz. El proyecto demuestra la capacidad de abstraer estructuras de datos complejas mediante Tipos de Datos Abstractos (TDA) propios e interfaces web funcionales.

Palabras clave

1. Objetos
2. Programación
3. Árboles
4. Jerarquía
5. Nodos

Abstract

This report details the design and development of a management and control system implemented in the C# programming language for the administration of a library's book catalog and categories. Utilizing the Object-Oriented Programming paradigm and the Model-View-Controller (MVC) pattern in ASP.NET Core, data loaded from XML format files is processed to construct the category hierarchy using an N-ary Tree based on the First Child / Next Sibling binary representation, without using the language's native collections. For the organization, querying, and maintenance of books within each category, an AVL Tree structure was implemented to guarantee efficient searches, insertions, and deletions through balancing rotations. Finally, the system generates visual reports in DOT code for the Graphviz tool. The project demonstrates the ability to abstract complex data structures through custom Abstract Data Types (ADT) and functional web interfaces.

Keywords

1. Objects
2. Programming
3. Trees
4. Hierarchy
5. Nodes

Introducción

En el ámbito de la ingeniería de software y la ciencia de la computación, la capacidad de modelar escenarios y sistemas de almacenamiento del mundo real por medio de estructuras de datos dinámicas es un pilar fundamental. Al adoptar la metodología de la programación orientada a objetos en el lenguaje C#, se da pie a la noción de tipos de datos abstractos, cuya implementación permite abstraer la complejidad interna del comportamiento del hardware y facilitar la interacción con los datos.

Este informe documenta detalladamente el diseño, la arquitectura interna y el desarrollo de un sistema de gestión para la administración del catálogo de libros y categorías de una biblioteca. El núcleo funcional del proyecto radica en la ingesta, lectura y procesamiento de información proveniente de archivos en formato XML, permitiendo construir en la memoria RAM una representación de la jerarquía de categorías mediante un árbol N-ario estructurado sobre la técnica binaria de primer hijo y siguiente hermano. Conforme ingresa la información, el sistema ramifica el árbol sin depender de arreglos estáticos o colecciones predefinidas. Adicionalmente, para la organización, consulta y mantenimiento de los elementos bibliográficos dentro de cada categoría, se diseñó e implementó una estructura de árbol binario de búsqueda auto-balanceado de tipo AVL. Esta combinación de estructuras se complementa con el patrón de arquitectura Modelo-Vista-Controlador en el entorno de ASP.NET Core y con la generación automática de reportes en código DOT para la herramienta Graphviz. A través de esta sinergia, el sistema evalúa la información, organiza los registros y determina las rutas de consulta más eficientes para el inventario.

1. Programación orientada a objetos

La programación orientada a objetos representa un paradigma de desarrollo en el cual la estructuración del software se fundamenta en torno a clases, objetos e instancias de datos, en contraposición al uso exclusivo de funciones globales y lógica secuencial. Su enfoque principal radica en las entidades que los programadores requieren manipular para reflejar el dominio del problema, en vez de centrarse únicamente en los algoritmos aislados necesarios para llevar a cabo dicha manipulación.

En este proyecto, los elementos fundamentales del dominio de la biblioteca —tales como Libro, Categoría, NodoCategoría y NodoAVL— se conceptualizan como entidades independientes. Cada una de estas clases encapsula un conjunto de atributos distintivos (como identificadores ISBN, títulos, autores, nombres de categorías y alturas de árbol) y

métodos de comportamiento orientados a la modificación y lectura de su propio estado interno.

1.1 Clase y Abstracción

Una clase funciona como una plantilla o molde sintáctico que establece las propiedades, estructuras y acciones que caracterizarán a una entidad dentro del sistema. La abstracción es el principio mediante el cual se seleccionan las características relevantes de un objeto del mundo real y se ignoran los detalles de implementación superfluos.

En la arquitectura del catálogo, la abstracción permite que las capas superiores del software, como las vistas HTML o las acciones del controlador, interactúen con las estructuras de datos únicamente a través de métodos públicos como agregar, buscar o eliminar. El usuario o el controlador solicita la inserción de un nuevo registro sin necesidad de conocer los algoritmos de asignación de punteros en la memoria dinámica o las operaciones de reestructuración de nodos. Esto facilita el mantenimiento de un código extenso, garantizando que el sistema opere con alta flexibilidad y bajo acoplamiento.

1.2 Encapsulamiento y Ocultamiento de Información

El encapsulamiento es el mecanismo de la programación orientada a objetos que restringe el acceso directo a algunos de los componentes de un objeto, protegiendo el estado interno contra modificaciones arbitrarias o no autorizadas. En la implementación de las estructuras de este proyecto, los punteros que conectan las jerarquías de categorías y los nodos de los árboles se manejan de manera privada o controlada a través de propiedades estrictas.

De este modo, la alteración del puntero hacia el primer hijo o hacia el siguiente hermano en el árbol N-ario solo puede ser ejecutada por la lógica interna de la clase ArbolCategorias. Del mismo modo, la modificación de la altura o de las ramas izquierda y derecha en el árbol AVL solo ocurre dentro de los métodos de balanceo de la clase ArbolAVL. Esto previene la corrupción de la estructura en memoria y garantiza la integridad de los datos durante todo el ciclo de ejecución de la aplicación.

2. Implementación en C# y Plataforma .NET

C# es un lenguaje de programación multiparadigma, fuertemente tipado, moderno y robusto, especialmente diseñado para el desarrollo de arquitecturas complejas y de alto rendimiento orientadas a objetos. En el marco de este proyecto,

C# y la plataforma ASP.NET Core permitieron la creación de una aplicación web bajo el patrón Modelo-Vista-Controlador (MVC), integrando la lógica interna de la memoria con una interfaz de usuario accesible a través del navegador.

El uso de C# facilitó la construcción desde cero de los Tipos de Datos Abstractos (TDA), permitiendo al programador gestionar manualmente el enlazado de referencias y la dirección de memoria de cada nodo. Esta implementación responde de manera estricta a las restricciones académicas fijadas para el desarrollo del software, las cuales prohíben categóricamente el uso de colecciones nativas del lenguaje o librerías del framework tales como List, LinkedList, Queue, Stack o arreglos dinámicos. La gestión manual de referencias garantiza que el estudiante comprenda la mecánica interna del manejo de punteros y la complejidad temporal de los algoritmos desarrollados.

3. Tipos de datos abstractos (TDA)

Un Tipo de Dato Abstracto (TDA) es un modelo matemático y lógico que define un conjunto de valores junto con las operaciones permitidas que pueden realizarse sobre ellos. La creación de un TDA es una práctica fundamental que provee un nivel de abstracción orientado a desacoplar el código que utiliza la estructura del código responsable de su implementación interna.

A diferencia de los tipos de datos primitivos provistos por los lenguajes de programación, los TDA desarrollados para este proyecto permiten modelar relaciones de jerarquía y de ordenamiento dinámico que crecen conforme la aplicación requiere almacenar más información.

3.1 Árbol N-Ario mediante Punteros Binarios

Un árbol N-ario es una estructura de datos no lineal en la cual cada nodo puede tener un número indeterminado de nodos descendientes o hijos. Dado que el número de hijos por categoría no es fijo y está sujeto al contenido de los archivos XML de entrada, la implementación tradicional mediante arreglos de punteros resulta ineficiente y contraviene la restricción de no usar estructuras nativas.

Para resolver este desafío, se implementó la técnica de representación binaria de árbol N-ario conocida como Primer Hijo / Siguierte Hermano. Bajo este esquema, cada nodo perteneciente a la clase NodoCategoría posee exactamente dos referencias o punteros de enlazado:

PrimerHijo: Un puntero vertical dirigido hacia abajo que apunta únicamente al primer elemento descendiente en la jerarquía.

SiguierteHermano: Un puntero horizontal dirigido hacia la derecha que conecta con el siguiente nodo que

comparte el mismo nivel jerárquico y el mismo nodo padre.

Esta arquitectura permite representar cualquier árbol con grado N utilizando únicamente una estructura de enlaces de tipo binario. El recorrido por las diferentes ramas del árbol de categorías se realiza mediante algoritmos de exploración recursiva en profundidad y en amplitud, lo que permite buscar una categoría específica, insertar nuevas secciones o verificar la existencia de duplicados sin desperdiciar recursos de memoria en punteros nulos.

3.2 Árbol Binario de Búsqueda Balanceado (AVL)

El Árbol AVL, bautizado así en honor a sus creadores Adelson-Velsky y Landis, es una estructura de datos jerárquica y dinámica de tipo Árbol Binario de Búsqueda (ABB) que incorpora la propiedad de mantenerse auto-balanceado de forma automática. En este proyecto, cada nodo del árbol N-ario correspondiente a una categoría contiene en su interior una instancia propia e independiente de un Árbol AVL. Esta estructura es la encargada de almacenar los libros pertenecientes a dicha sección. Un Árbol AVL cumple estrictamente con las propiedades de ordenamiento de un árbol binario de búsqueda: para cualquier nodo dado, todos los elementos ubicados en su subárbol izquierdo poseen un valor de clave menor (en este caso, el número de ISBN del libro), mientras que todos los elementos situados en su subárbol derecho poseen un valor de clave mayor.

La característica distintiva del Árbol AVL radica en su condición de equilibrio: para todo nodo dentro del árbol, la diferencia de alturas entre su subárbol izquierdo y su subárbol derecho (denominada Factor de Equilibrio) nunca puede ser superior a 1 ni inferior a -1. Esta propiedad asegura que la altura total del árbol se mantenga siempre dentro del orden logarítmico, garantizando que el tiempo requerido para buscar, insertar o eliminar un registro sea siempre óptimo.

3.3 Arquitectura y mecánica del balanceo

A diferencia de los árboles binarios de búsqueda simples, los cuales corren el riesgo de degenerar en una estructura lineal similar a una lista enlazada cuando los elementos son ingresados en orden ascendente o descendente, el Árbol AVL implementado analiza la topología de los nodos de manera constante.

Cada vez que se lleva a cabo una operación de inserción o eliminación de un libro, el algoritmo recursivo recalcula la altura de cada uno de los nodos involucrados en el camino de regreso hacia la raíz.

Posteriormente, evalúa el factor de equilibrio mediante la resta matemática de las alturas de sus ramas descendientes:

Factor de Equilibrio

$$= \text{Altura}(\text{Subárbol Izquierdo}) - \text{Altura}(\text{Subárbol Derecho})$$

Si al finalizar la operación el factor de equilibrio toma un valor igual a 2 o igual a -2, se determina que el nodo está desbalanceado y se procede a realizar una reestructuración de punteros conocida como rotación. Existen cuatro casos de rotaciones posibles dentro del algoritmo:

1. **Rotación Simple a la Derecha:** Se ejecuta cuando un nodo presenta un factor de equilibrio de +2 y su hijo izquierdo presenta una carga mayor hacia la izquierda (factor ≥ 0). Consiste en descender el nodo desbalanceado y elevar a su hijo izquierdo como la nueva raíz local.
2. **Rotación Simple a la Izquierda:** Se ejecuta cuando un nodo presenta un factor de equilibrio de -2 y su hijo derecho presenta una carga mayor hacia la derecha (factor ≤ 0). El nodo desbalanceado descende y su hijo derecho pasa a ocupar la posición superior.
3. **Rotación Doble Izquierda-Derecha:** Se aplica cuando un nodo tiene un factor de equilibrio de +2 pero su hijo izquierdo está cargado hacia la derecha (factor < 0). Se aplica primero una rotación izquierda sobre el hijo izquierdo y posteriormente una rotación derecha sobre el nodo padre desbalanceado.
4. **Rotación Doble Derecha-Izquierda:** Se aplica cuando un nodo tiene un factor de equilibrio de -2 pero su hijo derecho está cargado hacia la izquierda (factor > 0). Se aplica una rotación derecha sobre el hijo derecho y luego una rotación izquierda sobre la raíz desbalanceada.

Al finalizar las operaciones de reorientación de referencias, la propiedad de ordenamiento del número de ISBN se mantiene intacta y la altura del árbol vuelve a sus parámetros de equilibrio.

4. Algoritmos y Operaciones sobre el Catálogo

Para brindar una funcionalidad completa al usuario de la biblioteca, el sistema integra un conjunto de algoritmos encargados de procesar las consultas, adiciones y bajas dentro de las estructuras en memoria.

4.1 Búsqueda y Consultas por ISBN

La búsqueda de información sobre un libro específico se efectúa seleccionando la categoría destino y

ejecutando una exploración iterativa o recursiva sobre su Árbol AVL. El algoritmo inicia la comparación en el nodo raíz de la categoría: si el ISBN ingresado coincide con el ISBN del nodo actual, la búsqueda concluye con éxito devolviendo el objeto Libro. Si el valor es menor, el algoritmo se desplaza hacia la rama izquierda; de lo contrario, se desplaza hacia la derecha.

Adicionalmente, se desarrollaron métodos optimizados para la obtención del libro con menor ISBN y del libro con mayor ISBN almacenados en una categoría:

- **Obtener Menor ISBN:** Parte de la raíz del Árbol AVL y desciende de forma exclusiva a través de los punteros izquierdos hasta alcanzar el nodo hoja más alejado, el cual contiene por definición el valor mínimo del conjunto.
- **Obtener Mayor ISBN:** Parte de la raíz y desciende a través de los punteros derechos de la estructura hasta encontrar el nodo final, correspondiente al valor máximo almacenado.

4.2 Eliminación de Registros

El algoritmo de eliminación en el Árbol AVL localiza el nodo que contiene el ISBN a remover y evalúa tres escenarios posibles:

1. **El nodo es una hoja (sin hijos):** Se remueve la referencia directamente desde el nodo padre.
2. **El nodo tiene un solo hijo:** Se reemplaza el nodo a eliminar por la referencia de su único descendiente.
3. **El nodo tiene dos hijos:** Se localiza el sucesor en in-order (el nodo con el menor ISBN dentro del subárbol derecho), se copian sus datos hacia el nodo actual y se procede a eliminar de manera recursiva dicho sucesor.

Posteriormente a la eliminación, el algoritmo retrocede en la pila de llamadas recalculando la altura y aplicando las rotaciones necesarias para asegurar que la estructura permanezca balanceada.

5. Procesamiento de XML y Generación de Reportes con Graphviz

La persistencia temporal y la carga masiva de datos dentro del sistema se apoya en la interpretación de archivos planos estructurados en formato XML. Por otro lado, la verificación visual del estado de las estructuras se realiza mediante la herramienta Graphviz.

5.1 Interpretación de Archivos XML

El módulo de lectura procesa el flujo de datos proveniente del archivo subido por el usuario a través

de la interfaz web. La lectura se realiza en fases ordenadas para asegurar la integridad de las jerarquías:

1. **Lectura de Categorías:** Se extraen las etiquetas correspondientes a las categorías y se identifican sus atributos de dependencia o categoría padre. Si el atributo se encuentra presente, el algoritmo busca la categoría superior en el árbol N-ario para enlazar el nuevo nodo como su descendiente. Si la categoría padre aún no existe en la memoria, se inserta temporalmente como una raíz secundaria para evitar la pérdida de información y reordenar las ramas posteriormente.
2. **Lectura de Libros:** Se recorren las etiquetas de cada libro extrayendo el número de ISBN, título, autor y la categoría a la cual pertenece. El parser invoca al árbol N-ario para localizar el nodo de la categoría correspondiente e inserta de inmediato el nuevo libro dentro del Árbol AVL enlazado a dicha sección.

5.2 Generación de Código DOT y Visualización

Para cumplir con los requerimientos de la evaluación del curso, el sistema incorpora el servicio GraphvizService, el cual se encarga de recorrer la memoria interna y construir cadenas de texto estructuradas en el lenguaje script de Graphviz (DOT).

- **Reporte del Árbol N-Ario de Categorías:** Recorre la estructura mediante llamadas recursivas a los punteros PrimerHijo y SiguierteHermano. Imprime las conexiones dirigidas asignando etiquetas explícitas y colores diferenciados (azul para las relaciones de hijo y verde para las relaciones entre hermanos).
- **Reporte del Árbol AVL de Libros:** Toma la raíz del Árbol AVL de la categoría seleccionada y realiza un recorrido recursivo imprimiendo cada nodo en formato de registro HTML con el número de ISBN y el título. Cada enlace hacia la izquierda y hacia la derecha es dibujado identificando el balanceo y la ramificación exacta de la estructura.

El código DOT resultante es enviado a la vista de la aplicación dentro de un área de texto, permitiendo que el estudiante o evaluador copie dicho contenido y visualice el diagrama gráfico correspondiente en herramientas de renderizado en línea

Conclusiones

- La implementación de la técnica Primer Hijo /

SiguierteHermano para la construcción del árbol N-ario demostró ser una solución óptima y eficiente para modelar jerarquías de categorías sin un número fijo de descendientes. Esta arquitectura permitió representar la totalidad de la estructura del catálogo utilizando únicamente dos punteros por nodo, cumpliendo con las restricciones académicas de no hacer uso de colecciones nativas de C# y optimizando el consumo de memoria en el sistema.

- La utilización del Árbol AVL para la organización de los libros dentro de cada categoría garantizó tiempos de consulta, inserción y eliminación logarítmicos.
- El algoritmo de auto-balanceo mediante el cálculo constante del factor de equilibrio y la ejecución de rotaciones simples y dobles evitó de manera efectiva la degradación de la estructura hacia listas lineales, asegurando búsquedas eficientes por ISBN sin importar el orden en que se hayan ingresado los datos.
- El desacoplamiento logrado mediante la creación de Tipos de Datos Abstractos (TDA) propios facilitó la integración directa con el patrón de arquitectura Modelo-Vista-Controlador en ASP.NET Core. Esta abstracción permitió que la lógica de la interfaz web interactuara de forma limpia con los árboles en memoria, garantizando una navegación fluida entre consultas, registros y reportes sin exponer la complejidad interna del manejo de referencias dinámicas.
- El módulo de generación de código en sintaxis DOT para Graphviz demostró la correcta abstracción de la memoria RAM hacia representaciones visuales. A través de recorridos recursivos sobre el árbol N-ario

Referencias

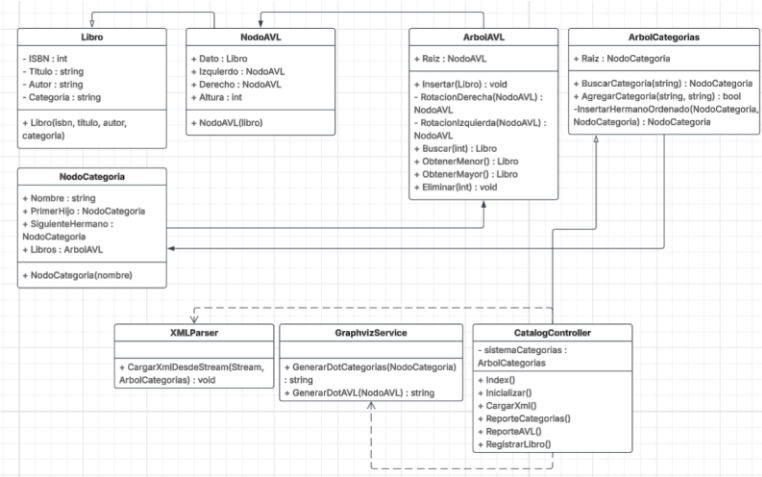
Martínez, M. (2 de noviembre de 2020). ¿Qué es la Programación Orientada a Objetos? Profile. <https://profile.es/blog/que-es-la-programacionorientada-a-objetos/>

Pérez, C. (18 de diciembre de 2020). Qué es y cómo abrir un archivo XML. MuyInteresante. <https://www.muyinteresante.es/tecnologia/23571.html>

Universidad Europea. (24 de agosto de 2022). Programación orientada a objetos: qué es y características. Universidad Europea. <https://universidadeuropea.com/blog/programacionorientada-objetos/>

Anexo

• Diagrama de Clases



• Diagrama de Actividades

Semana	Actividad
Semana 1	Definición de la arquitectura del proyecto en ASP.NET Core MVC, creación de las vistas/interfaz web principal y diseño de las clases base del modelo (Libro, NodoCategoria, NodoAVL)
Semana 2	Implementación de Tipos de Datos Abstractos TDA (Árbol N-Ario mediante Primer Hijo / Siguiente Hermano y Árbol AVL balanceado sin colecciones nativas). Desarrollo del parser XML para la ingesta de categorías y libros.
Semana 3	Desarrollo de la lógica de balanceo (rotaciones AVL), algoritmos de búsqueda por ISBN y eliminación de registros. Integración con GraphvizService para reportes .dot, pruebas de carga masiva y empaquetado del repositorio